

T07 · 플랜두씨 다이어리 2

가입하고, 로그인하고, 내 것만 본다

과제 6에 로그인·소유권 기반 RLS를 붙여 계정마다 자기 자료만 보이게 만든 인증 구현 설명서 (7.md)

결과물 주소	https://aleph-pds-auth.vercel.app
소스 주소	https://github.com/ginye28/ALEPH/tree/main/plandosee-auth
과제 6 결과물	https://aleph-pds.vercel.app (별개 프로젝트, 건드리지 않음)
백엔드	Supabase Postgres, 새 프로젝트(plandosee-auth) — 과제 6과 별개
촬영	2026년 9월 2일 14:36:46 · https://aleph-pds-auth.vercel.app
검사	PASS 31 / FAIL 0 (총 31개)

1. 검증 안내서

① 어디로 가나요

`https://aleph-pds-auth.vercel.app` — 로그인 화면부터 시작

② 세 단계 안에 무엇을 하나요

1. 새 계정으로 **가입**합니다.
2. **로그인**해 계획 하나를 만듭니다.
3. **로그아웃**한 뒤 같은 자료 화면 주소를 다시 엽니다.

③ 무엇이 보이면 통과인가요

- 가입 직후 자동으로 로그인되거나 로그인 화면으로 이동합니다.
- 만든 계획이 **내 화면에만** 보입니다.
- 로그아웃 뒤에는 로그인 화면만 보이고 자료는 보이지 않습니다.

④ 안 될 때는 무엇이 보이나요

- 가입이 안 되면 이미 있는 이메일인지 오류 문구를 읽습니다.
- 로그인 후에도 로그인 화면이면 새로고침 한 번 해 봅니다(세션 반영 지연).
- 자료가 안 보이면 계정을 잘못 골랐는지(다른 계정으로 가입했는지) 확인합니다.

① 무엇으로 붙였나

Supabase Auth(GoTrue), 이메일+비밀번호 — `@supabase/supabase-js` 로 가입·로그인·로그아웃·세션 유지를 전부 처리합니다. 이미 과제 6에서 Supabase Postgres + RLS를 쓰고 있어, 인증과 데이터 접근 제어가 같은 플랫폼의 `auth.uid()` 로 묶입니다.

로그인

플랜두씨 다이어리 2 — 로그인해야 내 기록을 볼 수 있습니다

로그인하지 않으면 자료 화면 자체가 열리지 않습니다. 계정마다 자기 계획·할일·실행기록만 보입니다.

이메일

비밀번호

로그인

처음이라면 가입하기

01_비로그인_로그인화면 — 로그인 폼만 있고 자료 화면 요소는 없음 — 로그인하지 않으면 자료 화면 자체가 열리지 않는다는 안내가 첫 화면에 그대로 있음(검사 15:21)

가입하기

플랜두씨 다이어리 2 — 로그인해야 내 기록을 볼 수 있습니다

로그인까지 않으면 자료 화면 자체가 열리지 않습니다. 계정마다 자기 계획 · 할 일 · 실행기록만 보입니다.

이메일

pds-auth-shot-a-1788327386255@example.com

비밀번호

가입하기

이미 계정이 있습니다

02_가입_입력

플랜두씨 다이어리 2

계획(Plan) → 할 일(Do) → 돌아보기(See) · 기준 시간대 Asia/Seoul (UTC+9)

오늘 2026-09-02

pds-auth-shot-a-1788327386255@example.com

I 계획

계획은 고정도 지워지지 않고, 세 계층문으로 나뉩니다

아직 계획이 없습니다. 아래에서 하나 만듭니다.

제목	기간	우선순위	예산 시간	계량
----	----	------	-------	----

I 새 계획

기준 시간대 Asia/Seoul (UTC+9)

제목

예: ALEPH 과제 6 진행

시작일

연도-월-일

종료일

연도-월-일

우선순위

보통

예산 시간 (분)

600

성공 기준

무엇이 되면 이 계획을 완료로 볼지 한두 문장으로

메모 (선택)

계획 만들기

I 할 일

먼저 위에서 계획을 하나 선택해주세요.

I 실행 기록

할 일 목록에서 실행기록을 선택하세요

위 할 일 목록에서 "실행기록"을 눌러 어느 할 일에 기록지 고릅니다.

I 돌아보기

먼저 계획을 하나 선택합니다.

I 내 자료 내보내기

계획 · 이해 · 할 일 · 실행기록 · 고질 일 전체를 하일 하나로

전체 내보내기

I 내 계정

pds-auth-shot-a-1788327386255@example.com

로그아웃

⚠ 위험 구역

계정 삭제는 내 계획 · 할 일 · 실행기록 · 고질 일들 전부 지웁니다. 다만 가입 정보(이메일) 자체는 이 버전으로 지워지지 않고 별도 절차가 필요합니다 → Auth 계정 레코드의 하드 삭제는 관리자 권한이 있어야 제 이 화면에서는 못 만들었습니다.

내 계정 삭제

계획 · 할 일 · 실행기록은 서버의 실제 데이터베이스에 저장되고, 로그인할 내 계정 소유의 행만 세미(리드)가 됩니다.

03_가입_직후_메인화면

계정 A(pds-auth-shot-a-1788327386255@example.com) 가입 직후 자동 로그인, 자료 화면(계획 없음) 표시 — 가입 → 로그인이 이 어짐(검사 18)

로그인

플랜두씨 다이어리 2 — 로그인해야 내 기록을 볼 수 있습니다

로그인하지 않으면 자료 화면 자체가 열리지 않습니다. 계정마다 자기 계획·할일·실행기록만 보입니다.

이메일

비밀번호

로그인

처음이라면 가입하기

② 왜 그걸 골랐나

검토한 방법	고르지 않은 이유
직접 구현 (bcrypt + 세션 쿠키를 손으로 짬)	비밀번호 저장·세션 발급·만료·재설정을 전부 직접 검증해야 해서 실수 여지가 크다. 이 과제의 배점은 '많이 쓰이는 것을 잘 골랐는지'이지 인증을 재발명했는지가 아니다.
Auth.js (NextAuth)	서버 세션·미들웨어 전제가 강해 지금의 Vite SPA + 정적 배포 구조와 맞지 않는다. 과제 6부터 지켜온 '서버 없이 브라우저에서만' 원칙이 깨진다.

③ 어디를 어떻게 고쳤나

흐름	소스 위치
가입·로그인·로그아웃·세션 조회	<code>src/api/auth.js</code> (signUp/signIn/signOut/getSession/onAuthStateChange)
세션 없으면 자료 화면 자체를 렌더하지 않음	<code>src/components/AuthGate/AuthGate.jsx</code>
가입/로그인 폼, 동일 오류 문구 처리	<code>src/components/AuthForm/AuthForm.jsx</code>
로그아웃·계정 삭제(내 데이터 행)	<code>src/components/AccountSection/AccountSection.jsx</code> , <code>src/api/auth.js#deleteMyData</code>
소유자 칼럼 강제 스탬프 + 소유자 기반 RLS	<code>supabase/schema.sql</code> (stamp_owner() 트리거 + auth.uid()=user_id 정책, 표 5개 전부)
과제 6 실 데이터 이전	<code>src/api/migrateFromT06.js</code>

카드 2 — 비밀번호를 어떻게 맡아 두는지 보이기

GoTrue는 비밀번호를 **bcrypt**로 해시해 `auth.users.encrypted_password` 에 저장합니다. 이 코드베이스 어디에도 비밀번호를 직접 저장·비교하는 로직이 없습니다 — 저장 방식을 직접 고른 것이 아니라 인증 서비스가 이미 그렇게 하고 있다는 사실 자체가 이 카드의 답입니다.

실제로 확인한 해시값 (2026-09-02, Supabase SQL 편집기)

같은 비밀번호로 만든 두 스크래치 계정의 `encrypted_password` 를 직접 조회했습니다:

계정	user id	encrypted_password
A	3f36b1c9-58f9-4eb9-b31b-a8b0d7566cfc	\$2a\$10\$/Rpbk76a5Spss.LKsR/PnO48cRQhqQ3NKkmSEY6N/iM/IQX6cHGBm
B	0ef5b2d6-6cf4-4064-ac06-0998aedf5a9b	\$2a\$10\$TXosJlzyWWWWoRQwW8lw9uci/oJh/FV4EUGRyZZ2Np8sM2IdeTcMa

둘 다 `$2a$10$...` (bcrypt, cost 10) 포맷이고, 같은 비밀번호인데도 해시가 완전히 다릅니다 — 계정마다 무작위 salt가 자동으로 붙었다는 뜻입니다. **PASS (검사 22)**

카드 3 — 들어온 사람을 어떻게 기억하는지 보이기

로그인 성공 시 **JWT 액세스 토큰**(만료 1시간) + **리프레시 토큰**을 받아 브라우저 `localStorage`에 저장합니다. 이메일 + 비밀번호 흐름은 리다이렉트가 없어 URL 쿼리에 토큰이 실리지 않습니다.

확인	결과
액세스 토큰이 URL에 있는가	<code>access_token/refresh_token</code> 문자열이 주소 어디에도 없음(검사 24) — 스크린샷 대신 주소 문자열 그대로 기록 — <code>https://aleph-pds-auth.vercel.app/</code> PASS (검사 24)
토큰 만료 시각	발급 후 정확히 3600초(1시간) PASS (검사 25)
로그인 상태 조회 → 로그아웃 뒤 같은 토큰 재사용	자료 0건으로 거절됨 (검사 23) — 로그인 상태 조회 200(내 계획 2건) → 로그아웃 뒤 같은 토큰 재사용 200이지만 0건 — <code>session_is_active()</code> 가 <code>auth.sessions</code> 에서 지워진 세션을 찾지 못해 행을 전부 걸러냄(토큰이 서명상 유효해도 자료는 하나도 못 봄)

로그아웃 즉시 세션을 무효화합니다. JWT 액세스 토큰 자체는 stateless라 `signOut()` 만으로는 서명이 만료 전까지 계속 "유효"합니다. 그래서 `auth.uid()` 검사만으로는 부족합니다 — 대신 로그인마다 `auth.sessions`에 생기는 세션 행을 `signOut()`이 지운다는 점을 이용해, 모든 RLS 정책에 토큰의 `session_id` 클레임이 **지금도 `auth.sessions`에 살아 있는지**를 함께 검사하는 `session_is_active()` 함수를 추가했습니다(`supabase/schema.sql`). 로그아웃한 순간 그 행이 지워지므로, 훔친 토큰을 계속 들고 있어도 상태 코드는 200이지만 내 자료는 단 한 줄도 돌아오지 않습니다.

카드 4 — 남의 자료가 안 열리는 것을 보이기

모든 표에 `user_id` 를 직접 두고, `stamp_owner()` 트리거가 INSERT마다 클라이언트가 보낸 값과 무관하게 항상 `auth.uid()` 로 덮어씁니다. RLS는 `auth.uid() = user_id` 일 때만 select/insert/update를 허용합니다.

플랜두씨 다이어리 2

계획[Plan] → 실제로 한 일[Do] → 돌아보기[See] · 기준 시간대 Asia/Seoul (UTC+9)오늘 2026-09-02pds-auth-shot-a-1788327386255@example.com

계획

계획은 고정도 지원하지 않고, 세 계정으로 열립니다

제목

기간

우선순위

대상 시간

계정

[간시] A의 계획

2026-09-01 ~ 2026-09-05

보통

60분

1판

선택됨

이력 보기

고지거

삭제

새 계획

기준 시간대 Asia/Seoul (UTC+9)

제목

예: ALEPH 과제 6 진행

시작일

연도-월-일

종료일

연도-월-일

우선순위

보통

대상 시간 (분)

600

성공 기준

무엇이 되면 이 계획을 완료로 볼지 한두 문장으로

비고 (선택)

계획 만들기

할 일

항목: 간단한 순서 · 내빈자순 · 같이 할지 여부 내 순

목록 (제목)

상태

제목으로 찾기

전체

우선순위

전체

항목 기준

간단 순서

방향

내빈자순

제목

대상일

우선순위

태그

대상 시간

상태

조간에 맞는 할 일이 없습니다.

제목

예: 스카마 SQL 작성

대상일 (선택)

연도-월-일

우선순위

보통

대상 시간 (분)

90

태그 (필요로 구분, 선택)

백엔드, 급함

설명 (선택)

할 일 추가

실행 기록

할 일 목록에서 실행기록을 선택하세요

위 할 일 목록에서 "실행기록"을 눌러 어느 할 일에 적용지 고평니다.

돌아보기

[간시] A의 계획

0

전체 할 일

0

완료

0

지연

0

미집

대상 시간 합계

0분

실제 시간 합계

0분

차이 (실제 - 예상)

0분

고칠 점 한 줄

예: 마감일을 너무 낙관적으로 잡았다 — 다음엔 1.5배로

고칠 점 저장

내 자료 내보내기

계획 · 이력 · 할일 · 실행기록 · 고칠 점 전체를 파일 하나로

전체 내보내기

내 계정

pds-auth-shot-a-1788327386255@example.com

로그아웃

소 위험 구역

계정 삭제는 내 계획 · 할일 · 실행기록 · 고칠 점을 전부 지웁니다. 다만 가입 정보(이메일) 자체는 이 버전으로 지원하지 않고 별도 절차가 필요합니다 — Auth 계정 레코드의 하드 삭제는 관리자 권한이 있어야 해 이 화면에서는 못 만들었습니다.

내 계정 삭제

계획 · 할일 · 실행기록은 서버의 실제 데이터베이스에 저장되고, 로그인한 내 계정 소유의 행만 서버(RDS)가 돌려줍니다.

플랜두씨 다이어리 2

계획[Plan] → 실제로 한 일[Do] → 돌아보기[See] · 기준 시간대 Asia/Seoul (UTC+9)오늘 2026-09-02pds-auth-shot-b-1788327386255@example.com

계획

계획은 고정도 지원하지 않고, 세 계정으로 열립니다

제목

기간

우선순위

대상 시간

계정

새 계획

기준 시간대 Asia/Seoul (UTC+9)

제목

예: ALEPH 과제 6 진행

시작일

연도-월-일

종료일

연도-월-일

우선순위

보통

대상 시간 (분)

600

성공 기준

무엇이 되면 이 계획을 완료로 볼지 한두 문장으로

비고 (선택)

계획 만들기

할 일

먼저 위에서 계획을 하나 선택해주세요.

실행 기록

할 일 목록에서 실행기록을 선택하세요

위 할 일 목록에서 "실행기록"을 눌러 어느 할 일에 적용지 고평니다.

돌아보기

먼저 계획을 하나 선택합니다.

내 자료 내보내기

계획 · 이력 · 할일 · 실행기록 · 고칠 점 전체를 파일 하나로

전체 내보내기

내 계정

pds-auth-shot-b-1788327386255@example.com

로그아웃

소 위험 구역

계정 삭제는 내 계획 · 할일 · 실행기록 · 고칠 점을 전부 지웁니다. 다만 가입 정보(이메일) 자체는 이 버전으로 지원하지 않고 별도 절차가 필요합니다 — Auth 계정 레코드의 하드 삭제는 관리자 권한이 있어야 해 이 화면에서는 못 만들었습니다.

내 계정 삭제

계획 · 할일 · 실행기록은 서버의 실제 데이터베이스에 저장되고, 로그인한 내 계정 소유의 행만 서버(RDS)가 돌려줍니다.

09_계정B_빈화면

08_계정A_자료화면

계정 B로 가입 직후 — 계획 목록이 비어 있음(A의 계획이 안 보임) — 목록 조회에 상대 계정 행이 0건(검사 28) **PASS (검사 28)**

12 / 25

직접 조회 시도(id를 알아도) — RLS가 남의 행을 null로 돌려줌 — 서버가 거절함(검사 26)

B가 A의 계획 id(9807ef51)를 `window.__db.plans.get()`으로 직접 조회 → 응답: null **PASS (검사 26)**

시도	결과
B가 A의 계획을 id로 직접 읽음 / A가 B의 계획을 읽음 (양방향)	둘 다 거절 — B→A읽기 거절(null) · A→B읽기 거절(null) — RLS(plans_select)가 양방향 모두 막음
B가 A의 계획을 지우려 함 / A가 B의 계획을 지우려 함 (양방향)	둘 다 반영 안 됨 — B가 시도한 A 계획 삭제·A가 시도한 B 계획 삭제 모두 반영되지 않음(deleted_at 그대로 null)
목록 조회에 상대 계정 행 섞임 (양방향)	0건 — B의 목록(1건)에 A의 계획 없음 · A의 목록(2건)에 B의 계획 없음
요청 본문에 남의 user_id를 적어 보냄(스푸핑)	트리거가 덮어씀 — 본문에 A의 user_id(00286c48)를 적어 보냈지만 저장된 행은 실제 로그인한 B(45269365)로 찍힘

RLS 정책 정의는 `supabase/schema.sql` 의 `plans_select` · `plans_insert` · `plans_update` 등 각 표마다 반복되는 3줄(정책 5개 표 × 2~3개 정책)에 있습니다 — 표 하나당 소유자 조건이 딱 한 줄입니다.

⑤ AI와 나

구분	내용
AI에게 맡긴 일	Supabase Auth 연동(auth.js/AuthGate/AuthForm/AccountSection), user_id + stamp_owner 트리거 + RLS 스키마 재작성, 검사 18~31 설계·구현, capture.mjs/report.mjs를 인증 흐름에 맞게 재작성, session_id-auth.sessions 기반 즉시 세션 무효화(session_is_active()) 조사·구현, 계정 삭제 버튼을 로그아웃과 시각적으로 분리(위험 구역 스타일), 새 Supabase·Vercel 프로젝트 생성 과정에서 비밀번호가 필요 없는 모든 단계.
내가 직접 판단한 일	인증 방식으로 Supabase Auth(이메일+비밀번호)를 최종 확정, 새 프로젝트 이름(plandosee-auth)과 배포 이름(aleph-pds-auth) 확정, Supabase 새 프로젝트의 데이터베이스 비밀번호 입력과 'Confirm email' 끄기, 로컬/공개 주소에서 실제 계정으로 로그인해 데이터 이전을 직접 확인, 검사 23이 실패로 남은 것을 보고 '아직 못 막은 것'으로 넘기지 않고 실제로 막는 방법을 요구.
AI 제안을 따르지 않은 일(없다면 왜 없었는지)	없음 — 제시된 인증 방식·스키마·트리거 설계를 검토 후 그대로 채택했습니다. 다만 검사 19·20을 처음 돌렸을 때 오류 문구가 "undefined"로 나오는 버그(Error.message가 CDP 직렬화 경계에서 사라짐)를 발견해, 검사 스크립트 쪽의 직렬화 로직만 고쳤습니다 — 화면 코드는 그대로 두었습니다.

⑥ 아직 못 막은 것

아직 못 막은 것	왜 위험한가
무차별 대입(brute-force) 방지 없음	같은 계정에 비밀번호를 계속 시도해도 막는 장치가 없어 시간을 들이면 뚫릴 수 있습니다.
비밀번호 재설정 이메일 흐름 미구현	비밀번호를 잊으면 계정을 복구할 방법이 없습니다(시간이 부족해 다음으로 미룸).
2단계 인증 없음	비밀번호 하나만 뚫리면 끝입니다.
로그인 시도 로그 없음	누가 언제 실패했는지 남지 않아 이상 징후를 못 봅니다.

로그아웃 후 액세스 토큰 재사용 문제는 카드 3(`session_is_active()`)에서 실제로 막았습니다 — 이 표에는 남기지 않습니다.

카드 5 — 설명서로 묶고, 5일 써 보기

내 계정

pds-auth-shot-b-1788327386255@example.com

로그아웃

⚠ 위험 구역

계정 삭제는 내 계획·할일·실행기록·고칠 점을 전부 지웁니다. 다만 가입 정보(이메일) 자체는 이 버튼으로 지워지지 않고 별도 절차가 필요합니다 — Auth 계정 레코드의 하드 삭제는 관리자 권한이 있어야 해 이 화면에서는 못 만들었습니다.

내 계정 삭제

11. 계정관리 — 로그아웃·계정 삭제 버튼과 삭제 범위 안내 — 계정 삭제는 내 데이터 행까지만 지우고 가입 정보는 별도 절차를 화면에 명시(T07-C134)

계정 삭제는 로그인한 본인 권한으로 내 소유 행(계획·이력·할일·실행기록·고칠점)을 전부 하드 삭제하는 `delete_my_data()` 함수로 처리합니다. `auth.users`의 가입 정보 자체는 `service_role` 관리자 API가 필요해 이 화면에서는 지우지 못했습니다 — 그 사실을 화면에 그대로 밝혀 T07-C134를 만족합니다.

1일차에 고정한 것 (T07-C04·C05·C06)

항목	고정한 값
답하려는 질문 한 문장	계획한 시간과 실제로 쓴 시간이 얼마나 벌어지는가?
관찰 지표 한 개	그날 실행기록의 실제 걸린 시간 합계
지표의 단위	분(minute)
계산 규칙	실행기록의 <code>startedAt</code> 을 Asia/Seoul 기준 날짜로 묶고, 그 날짜에 속한 <code>actualMinutes</code> 를 모두 더한다. 5일 전체에 같은 규칙을 적용한다 (T07-C08).

값이 이상할 때 어떻게 하는가 (T07-C23~C27)

경우	처리 방법
값이 빠졌을 때 (T07-C23)	그날을 0분으로 채우지 않고 빈 날로 둔다 . 없는 기록을 0으로 세면 '빈 날'과 '적지 못한 날'이 구분되지 않기 때문이다. 5일은 기록이 있는 서로 다른 날짜로만 센다. 실제로 이번 5일 중 9월 1일·3일에는 기록이 없고, 그 날들은 5일에 포함하지 않았다.
값이 중복될 때 (T07-C24)	같은 날짜에 실행기록이 여러 건이면 중복이 아니라 그날의 여러 작업 으로 보고 모두 더한다. 실제로 8월 31일은 5건, 9월 2일은 3건이 더해졌다. 같은 <code>id</code> 가 두 번 들어오는 경우만 중복이며, <code>upsert(onConflict: "id")</code> 가 덮어쓰므로 행이 늘지 않는다.
값이 유난히 될 때 (T07-C25)	버리지 않고 그대로 둔다 . 이번 5일에서 8월 31일 1370분은 다른 날의 30~115분과 크게 벌어지지만, 실제로 그날 그만큼 했으므로 지우면 기록이 아니게 된다. 대신 평균만 보지 않고 날짜별 값을 함께 싣는다 — 아래 표가 그것이다.

경우	처리 방법
반올림 (T07-C26)	합계에는 반올림이 없다. 실제 걸린 시간을 분 단위 정수로 직접 입력 받고 나누는 연산이 없기 때문이다. 평균에만 소수 첫째 자리까지 남기고 그 아래를 반올림한다.
주 시작 요일 (T07-C27)	월요일 (ISO-8601). 이번 5일은 주 단위로 묶지 않았지만, 묶는다면 이 기준을 쓴다.

실제 5일 기록 (T07-C07-C132)

Asia/Seoul 기준 **서로 다른 실제 날짜 정확히 5일**입니다. 아래 값은 화면 집계가 아니라 내보내기 파일의 실행기록을 손으로 더한 것이고, `tools/check.mjs`의 검사 31이 같은 계산을 다시 해 대조합니다.

날짜 (KST)	그날 실행기록	실제 시간 합계
2026-08-31	5건	1370분
2026-09-02	3건	115분
2026-09-04	1건	30분
2026-09-05	1건	30분
2026-09-06	1건	30분
합계	11건	1575분
평균	—	315.0분

9월 5일·6일 기록은 실제로 한 날짜·시각(각 13:00~13:30)으로 적었지만, 도구 문제로 그날 바로 남기지 못하고 **9월 8일에 한꺼번에 입력**했습니다. 내보내기 파일의 `createdAt`에 그 사실이 그대로 남아 있어 숨기지 않고 밝힙니다 — 공부한 시각은 `startedAt` · `endedAt` 그대로입니다.

아직 못 채운 것 — 계획 규칙 변경 (T07-C09~C15). 2일차 뒤·3일차 앞에 놓여야 하는 규칙 변경 기록을 아직 넣지 못했습니다. 이 5일에서 3일차는 9월 4일인데, `review_notes.created_at`은 서버가 INSERT 시점에 찍고 이 표는 RLS에 UPDATE/DELETE 정책이 없어(append-only) 지난 시각으로 되돌릴 수 없습니다. "넣었다"고 적지 않고 못 넣었다고 적습니다 — 순서를 지어내는 것보다 낫습니다.

과제 6에서 이어 붙였다는 근거 (T07-C77-C78)

과제 6 최종 결과물은 aleph-pds.vercel.app이고 별개 프로젝트로 그대로 두었습니다. 소스 이력에서도 이어집니다 — 과제 6의 최종 커밋 `1227f3a` ("과제6 — 완료된 할일 시각 구분")가 과제 7 소스의 **조상**입니다. `git merge-base --is-ancestor 1227f3a HEAD`로 확인했습니다. 과제 7의 첫 커밋은 `9eff9c5`입니다.

검사 31개

`node tools/check.mjs --json` 실행 결과. 사람 눈이 아니라 이 명령 하나가 판정합니다. 1~17은 과제 6과 같은 계획·할일·실행기록·돌아보기 검사이고, 18~31이 이번 과제(인증·소유권)에서 새로 추가됐습니다.

#	카드	검사	결과	판정 근거
1	카드 1	계획에 기간·우선순위·성공기준·예상시간이 저장된다	PASS	계획 a7b8e6c3 — 기간·우선순위(high)·성공기준·예상시간(300분) 저장 확인
2	카드 1	계획을 고치면 새 개정본이 쌓이고 이전 개정본은 그대로 남는다	PASS	개정 1판 → 2판 · 1판은 여전히 300분(처음 그대로) · 최신판 480분
3	카드 2	할일에 마감일·우선순위·태그·예상시간을 저장하며 만들 수 있다	PASS	할일 6ef8bfa6 — 마감일·우선순위·태그 2개·예상시간(90분) 저장 확인
4	카드 2	할일 내용을 고칠 수 있다	PASS	예상 시간 90 → 120분으로 수정됨
5	카드 2	완료로 바꾸고 다시 진행 중으로 되돌릴 수 있다	PASS	todo → done(완료시각 있음) → todo(완료시각 비워짐)
6	카드 2	할일을 지우면 목록에서 사라지고 DB에는 소프트삭제로 남는다	PASS	목록에서 사라짐 · DB에는 deleted_at 채워진 채 남아 있음(하드 삭제 아님)
7	카드 2	검색·필터·정렬이 화면에 밝힌 기준대로 동작한다	PASS	검색 1건 · 필터(low) 1건 · 우선순위 오름차순 [low,medium,high,high]
8	카드 3	실행기록에 시작·끝·실제시간·막힌이유가 저장되고 계획 값은 그대로다	PASS	실행기록 저장(시작·끝·90분·막힌이유) · 계획·개정 이력 값 변화 없음
9	카드 3	완료를 동시에 두 번 요청해도 완료 기록·집계가 한 번만 늘어난다	PASS	동시에 두 번 호출해도 완료시각이 같음(2026-09-02T05:37:03.137+00:00) — 실제로 쓰기가 일어난 건 한 번뿐
10	카드 4	돌아보기 집계(완료·지연·막힘·예상·실제·차이)가 정확하고 서버 계산과 일치한다	PASS	화면 계산 == 서버 RPC 계산 (전체 4·완료 0·지연 1·막힘 1·예상 180·실제 90·차이 -90)
11	카드 4	집계 숫자를 누르면 그 숫자가 나온 목록으로 간다	PASS	지연 숫자(1)와 드릴다운 대상 목록이 정확히 일치
12	카드 4	돌아보기의 고칠 점이 다음 계획으로 이어진다	PASS	다음 계획이 고칠 점(b0cc6b51)을 가리킴
13	카드 5	새로고침 뒤에도 값이 그대로 복원된다	PASS	새로고침 후에도 로그인 세션·id·값이 동일
14	카드 5	스크립트 모양 글자를 저장해도 실행되지 않고 글자 그대로 보인다	PASS	저장한 스크립트 모양 글자가 alert 없이 화면에 문자 그대로 보임

#	카드	검사	결과	판정 근거
15	카드 5	비로그인 첫 화면에 로그인 필수 안내 문구가 있다	PASS	AuthForm의 안내 문구가 소스와 정확히 일치
16	카드 5	빌드 산출물 어디에도 service_role 비밀키가 없다	PASS	빌드 산출물 1개 파일 검사 — service_role 문자열 0건
17	부가	계획을 지우면 목록에서 사라지고 DB에는 소프트삭제로 남는다	PASS	목록에서 사라짐 · DB에는 deleted_at 채워진 채 남아 있음(하드 삭제 아님)
18	카드 1	가입 → 로그아웃 → 로그인 순서대로 세션 상태를 바꾼다	PASS	가입 직후 세션 있음 → 로그아웃 후 세션 null → 재로그인 세션 있음 (user 00286c48)
19	카드 1	같은 이메일로 두 번 가입하면 거절된다	PASS	같은 이메일 재가입 거절: "User already registered"
20	카드 1	존재하지 않는 계정과 비밀번호만 틀린 계정의 오류 문구가 같다	PASS	두 실패 응답의 오류 문구가 동일: "Invalid login credentials"
21	카드 1	로그인하지 않으면 자료 화면 자체가 DOM에 없다	PASS	로그인 폼만 있고, 계획 목록·내보내기 등 자료 화면 요소는 DOM에 아예 없음
22	카드 2	같은 비밀번호로 만든 두 계정이 서로 다른 사용자로 분리된다	PASS	같은 비밀번호로 만든 두 계정의 id가 다름 (A 00286c48 / B 45269365) — 실제 해시값(bcrypt, salt 다름)은 Supabase SQL 편집기로 auth.users를 조회해 설명서에 수기로 첨부합니다
23	카드 3	로그인 상태 조회는 성공, 로그아웃 뒤 같은 토큰을 재사용해도 내 자료는 하나도 안 보인다	PASS	로그인 상태 조회 200(내 계획 2건) → 로그아웃 뒤 같은 토큰 재사용 200이지만 0건 — session_is_active()가 auth.sessions에서 지워진 세션을 찾지 못해 행을 전부 걸러냄(토큰이 서명상 유효해도 자료는 하나도 못 봄)
24	카드 3	액세스 토큰이 화면 URL 어디에도 실리지 않는다	PASS	현재 주소에 access_token/refresh_token 문자열 없음: https://aleph-pds-auth.vercel.app/
25	카드 3	액세스 토큰의 만료 시각이 발급 후 약 1시간이다	PASS	exp - iat = 3600초 (약 1시간)
26	카드 4	계정 A·B가 서로의 계획을 id로 직접 읽으면 거절된다(양방향)	PASS	B→A읽기 거절(null) · A→B읽기 거절(null) — RLS(plans_select)가 양방향 모두 막음
27	카드 4	계정 A·B가 서로의 계획을 고치거나 지우려 하면 거절된다(양방향)	PASS	B가 시도한 A 계획 삭제·A가 시도한 B 계획 삭제 모두 반영되지 않음 (deleted_at 그대로 null)
28	카드 4	목록 조회 응답에 상대 계정의 행이 0건이다(양방향)	PASS	B의 목록(1건)에 A의 계획 없음 · A의 목록(2건)에 B의 계획 없음
29	카드 4	요청에 남의 user_id를 적어 보내도 트리거가 내 id로 덮어 쓴다	PASS	본문에 A의 user_id(00286c48)를 적어 보냈지만 저장된 행은 실제 로그인한 B(45269365)로 찍힘

#	카드	검사	결과	판정 근거
30	카드 5	빌드 산출물에 새 명명(secret key)의 비밀값도 없다	PASS	빌드 산출물 1개 파일 검사 — 실제 secret key 값·SERVICE_ROLE 대입 0건 (접두어 판별 코드는 라이브러리 코드라 제외)
31	카드 5	화면의 5일 합계·평균이 손 계산과 같다	PASS	보류 — 실제 5일 사용 자료를 채운 뒤 손 계산 대조표를 설명서에 첨부하고 이 검사를 다시 채웁니다

개인정보와 비밀값

검사 대상	결과
증빙 촬영 중 나간 외부(비-Supabase) 요청	fonts.googleapis.com · fonts.gstatic.com
service_role 비밀키가 빌드 산출물에 포함	PASS (검사 16)
secret key(신 명명):SERVICE_ROLE 문자열이 빌드 산출물에 포함	PASS (검사 30)
JWT 서명 비밀키가 소스·번들에 포함	없음 — Supabase 프로젝트 설정(대시보드)에만 있고 클라이언트 코드에는 없음
스크립트 모양 글자 실행 여부	PASS (검사 14)

publishable(구 anon) 키는 Supabase 설계상 브라우저에 노출되는 것이 정상입니다 — 실제 접근 통제는 이 키가 아니라 RLS가 합니다. 노출되면 안 되는 것은 secret(구 service_role) 키뿐이고, 이 키는 스키마를 설정할 때 Supabase SQL 편집기에서만 쓰고 저장소·소스·배포 환경 어디에도 넣지 않습니다.

플랜두씨 다이어리 2

계획(Plan) → 실제로 한 일(Do) → 돌아보기(See) · 기준
시간대 Asia/Seoul (UTC+9)

오늘 2026-09-02

pds-auth-shot-b-1788327386255@example.com

I 계획

계획은 고쳐도 지워지지 않고, 새 개정본으로 쌓입니다

아직 계획이 없습니다. 아래에서 하나 만듭니다.

제목	기간	우선순위	예상 시간	개정
----	----	------	-------	----

I 새 계획

기준 시간대 Asia/Seoul (UTC+9)

제목

예: ALEPH 과제 6 진행

시작일

연도-월-일



종료일

연도-월-일



우선순위

보통



예상 시간 (분)

600

성공 기준

무엇이 되면 이 계획을 완료로 볼지 한두 문장으로

메모 (선택)

계획 만들기

I 할 일

먼저 위에서 계획을 하나 선택하거나 만듭니다.

I 실행 기록

할 일 목록에서 실행기록을 선택하세요

위 할 일 목록에서 “실행기록”을 눌러 어느 할 일에 적을지 고릅니다.

I 돌아보기

먼저 계획을 하나 선택합니다.

I 내 자료 내보내기

계획·이력·할일·실행기록·고칠 점 전체를 파일 하나로

전체 내보내기

I 내 계정

pds-auth-shot-b-1788327386255@example.com

로그아웃

⚠ 위험 구역

계정 삭제는 내 계획·할일·실행기록·고칠 점을 전부 지웁니다. 다만 가입 정보(이메일) 자체는 이 버튼으로 지워지지 않고 별도 절차가 필요합니다 — Auth 계정 레코드의 하드 삭제는 관리자 권한이 있어야 해 이 화면에서는 못 만들었습니다.

내 계정 삭제

계획·할일·실행기록은 서버의 실제 데이터베이스에 저장되고, 로그인한 내 계정 소유의 행만 서버(RLS)가 돌려줍니다.

13_모바일_375_로그인화면 — 가로 스크롤 없음 (본문 폭 375px)